

Kruskal-Wallis - Examples in R

B.A. Hartlaub and A.S. Wagaman

One-Way Layout Example

Data for First Example

We look at some data about airquality from New York from May to September 1973, contained in R.

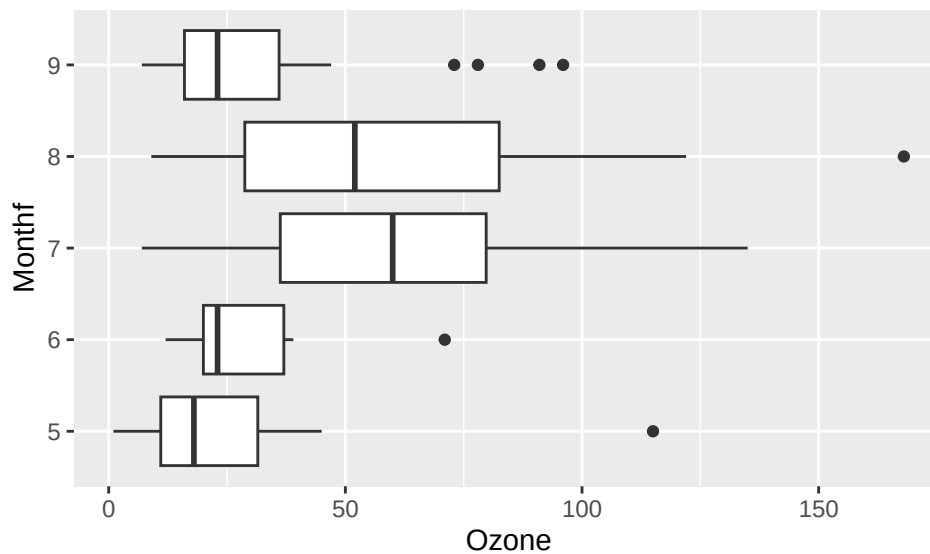
```
data(airquality)
names(airquality) # what variables are present

## [1] "Ozone" "Solar.R" "Wind" "Temp" "Month" "Day"

# make a factor variable for groups
airquality <- mutate(airquality, Monthf = factor(Month))

ggplot(aes(x = Ozone, y = Monthf), data = airquality) + geom_boxplot()
```

```
## Warning: Removed 37 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```

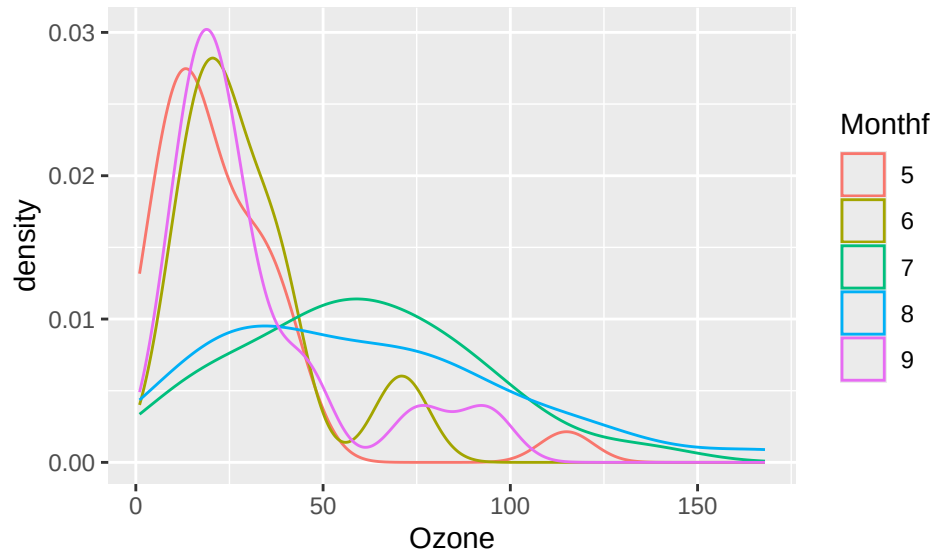


```
favstats(Ozone ~ Monthf, data = airquality)
```

##	Monthf	min	Q1	median	Q3	max	mean	sd	n	missing
## 1	5	1	11.00	18	31.50	115	23.61538	22.22445	26	5
## 2	6	12	20.00	23	37.00	71	29.44444	18.20790	9	21
## 3	7	7	36.25	60	79.75	135	59.11538	31.63584	26	5
## 4	8	9	28.75	52	82.50	168	59.96154	39.68121	26	5
## 5	9	7	16.00	23	36.00	96	31.44828	24.14182	29	1

```
ggplot(aes(x = Ozone, color = Monthf), data = airquality) + geom_density()
```

```
## Warning: Removed 37 rows containing non-finite outside the scale range
## (`stat_density()`).
```



What conditions would you worry about being violated here (for ANOVA or the Kruskal-Wallis procedure?)

Running a Parametric ANOVA in R

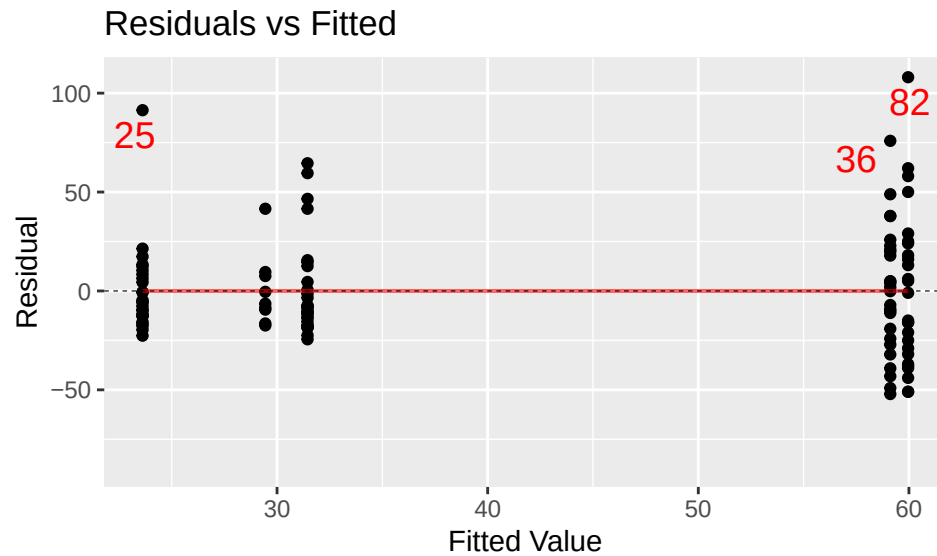
```
mod <- lm(Ozone ~ Monthf, data = airquality)
anova(mod)
```

```
## Analysis of Variance Table
##
## Response: Ozone
##          Df Sum Sq Mean Sq F value    Pr(>F)
## Monthf      4  29438   7359.5   8.5356 4.827e-06 ***
## Residuals 111  95705    862.2
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

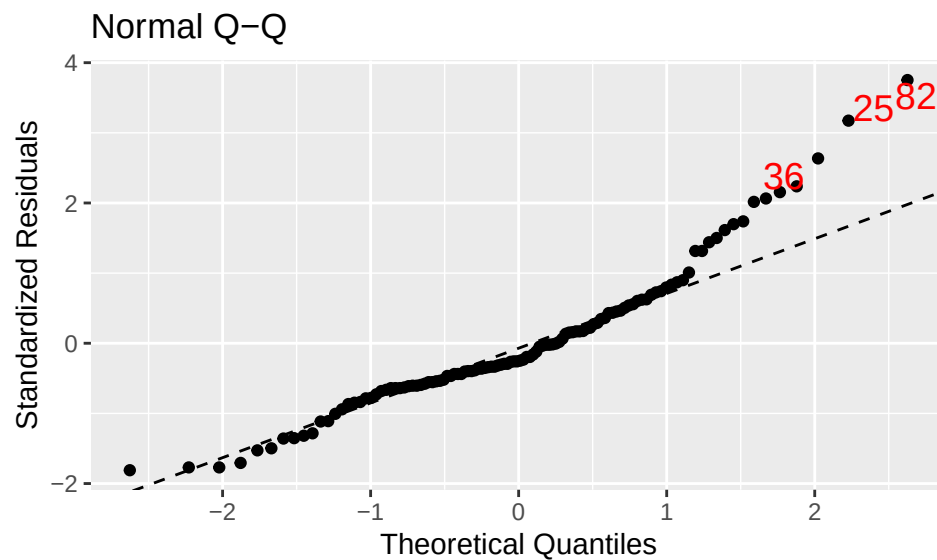
What are the test statistic and p-value here?

```
mplot(mod, which = 1) # checking for equal variance
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
mplot(mod, which = 2) # checking normality condition, using residuals
```



```
# equivalent to checking each group for normality
```

Noting any concerns about the analysis, what would your conclusion be here?

Parametric Multiple Comparisons

Here are examples of some different multiple comparisons procedures in the parametric case and how to run them.

Fisher's LSD

```
with(airquality, pairwise.t.test(Ozone, Monthf, p.adj = 'none'))
```

```
##
## Pairwise comparisons using t tests with pooled SD
```

```
##
## data:  Ozone and Monthf
##
##      5      6      7      8
## 6 0.60877 -      -      -
## 7 2.9e-05 0.01023 -      -
## 8 1.9e-05 0.00831 0.91744 -
## 9 0.32545 0.85838 0.00070 0.00048
##
## P value adjustment method: none
```

This provides just p-values for the associated comparisons, and while the p-value adjustment method is set to “none” that appears to mean Fisher’s LSD. P-values $\leq$ your selected overall alpha indicate a significant difference in means. This is the most liberal method of the three here. We do not recommend using this procedure for multiple comparisons. We show it here in the examples just so you see this option does exist.

Bonferroni

```
with(airquality, pairwise.t.test(Ozone, Monthf, p.adj = 'bonf'))
```

```
##
## Pairwise comparisons using t tests with pooled SD
##
## data:  Ozone and Monthf
##
##      5      6      7      8
## 6 1.00000 -      -      -
## 7 0.00029 0.10225 -      -
## 8 0.00019 0.08312 1.00000 -
## 9 1.00000 1.00000 0.00697 0.00485
##
## P value adjustment method: bonferroni
```

Again, this provides just p-values for the relevant comparison. Bonferroni is the most conservative of the methods shown here.

Tukey’s HSD

```
TukeyHSD(mod) #just feed it the model
```

```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = x)
##
## $Monthf
##      diff      lwr      upr      p adj
## 6-5  5.8290598 -25.6630930 37.321213 0.9858842
## 7-5 35.5000000 12.9158068 58.084193 0.0002795
## 8-5 36.3461538 13.7619606 58.930347 0.0001869
## 9-5  7.8328912 -14.1594735 29.825256 0.8603562
## 7-6 29.6709402 -1.8212127 61.163093 0.0749128
## 8-6 30.5170940 -0.9750588 62.009247 0.0622826
## 9-6  2.0038314 -29.0666371 33.074300 0.9997676
## 8-7  0.8461538 -21.7380394 23.430347 0.9999733
```

```
## 9-7 -27.6671088 -49.6594735 -5.674744 0.0061535
## 9-8 -28.5132626 -50.5056273 -6.520898 0.0043387
```

This actually shows you the confidence intervals and the p-values (adjusted appropriately) with the 95% family-wise confidence level. It is a compromise method between the other two, and we recommend using this method when possible.

Alternative package with all three functions

You may want to consider this option for the adjusted p-values for parametric multiple comparisons.

```
# requires DescTools package, loaded above
PostHocTest(aov(Ozone ~ Monthf, data = airquality), method = "lsd") # Fisher's LSD
```

```
##
## Posthoc multiple comparisons of means : Fisher LSD
## 95% family-wise confidence level
##
## $Monthf
##      diff      lwr.ci      upr.ci      pval
## 6-5  5.8290598 -16.673970  28.33209 0.60877
## 7-5  35.5000000  19.362241  51.63776 2.9e-05 ***
## 8-5  36.3461538  20.208395  52.48391 1.9e-05 ***
## 9-5   7.8328912  -7.881971  23.54775 0.32545
## 7-6  29.6709402   7.167910  52.17397 0.01023 *
## 8-6  30.5170940   8.014064  53.02012 0.00831 **
## 9-6   2.0038314 -20.197880  24.20554 0.85838
## 8-7   0.8461538 -15.291605  16.98391 0.91744
## 9-7 -27.6671088 -43.381971 -11.95225 0.00070 ***
## 9-8 -28.5132626 -44.228125 -12.79840 0.00048 ***
##
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
PostHocTest(aov(Ozone ~ Monthf, data = airquality), method = "hsd") # Tukey's HSD
```

```
##
## Posthoc multiple comparisons of means : Tukey HSD
## 95% family-wise confidence level
##
## $Monthf
##      diff      lwr.ci      upr.ci      pval
## 6-5  5.8290598 -25.6630930  37.321213 0.98588
## 7-5  35.5000000  12.9158068  58.084193 0.00028 ***
## 8-5  36.3461538  13.7619606  58.930347 0.00019 ***
## 9-5   7.8328912 -14.1594735  29.825256 0.86036
## 7-6  29.6709402  -1.8212127  61.163093 0.07491 .
## 8-6  30.5170940  -0.9750588  62.009247 0.06228 .
## 9-6   2.0038314 -29.0666371  33.074300 0.99977
## 8-7   0.8461538 -21.7380394  23.430347 0.99997
## 9-7 -27.6671088 -49.6594735 -5.674744 0.00615 **
## 9-8 -28.5132626 -50.5056273 -6.520898 0.00434 **
##
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
PostHocTest(aov(Ozone ~ Monthf, data = airquality), method = "bonf") # Bonferroni
```

```
##
## Posthoc multiple comparisons of means : Bonferroni
## 95% family-wise confidence level
##
## $Monthf
##          diff      lwr.ci      upr.ci      pval
## 6-5    5.8290598 -26.697640  38.355760 1.00000
## 7-5   35.5000000  12.173895  58.826105 0.00029 ***
## 8-5   36.3461538  13.020048  59.672259 0.00019 ***
## 9-5    7.8328912 -14.881944  30.547726 1.00000
## 7-6   29.6709402  -2.855760  62.197640 0.10225
## 8-6   30.5170940  -2.009606  63.043794 0.08312 .
## 9-6    2.0038314 -30.087331  34.094994 1.00000
## 8-7    0.8461538 -22.479952  24.172259 1.00000
## 9-7 -27.6671088 -50.381944  -4.952274 0.00697 **
## 9-8 -28.5132626 -51.228097  -5.798428 0.00485 **
##
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Running the Kruskal Wallis Test

First, we can look up the cutoff for the test statistic H based on the sample sizes in the groups.

```
cutoff <- cKW(0.05, c(26, 9, 26, 26, 29))
cutoff
```

```
##
## Monte Carlo Approximation (with 10000 Iterations) used:
##
## Group sizes: 26 9 26 26 29
## For the given alpha=0.05, the upper cutoff value is Kruskal-Wallis H=9.45651929,
## with true alpha level=0.05
```

Note that you don't need those sample sizes in any particular order, and the output may differ slightly due to Monte Carlo approximations used, unless you set a seed. In other words, it is simulating the null distribution for us.

```
# re-running the entire chunk will generate the same H cutoff value
set.seed(225)
cKW(0.05, c(9, 26, 26, 26, 29)) # arbitray order: least to greatest for sample sizes
```

```
##
## Monte Carlo Approximation (with 10000 Iterations) used:
##
## Group sizes: 9 26 26 26 29
## For the given alpha=0.05, the upper cutoff value is Kruskal-Wallis H=9.47702843,
## with true alpha level=0.05
```

In order to use pKW to find p-values, we can have the data in one of two formats. From the help menu, we learn that the data entry is intended to be flexible, so that the groups of data can be entered in either of two ways.

For data with 2 groups, $a = 1, 2$ and $b = 3, 4, 5$ the following are equivalent:

```
pKW(x = list(c(1, 2), c(3, 4, 5)))
```

```
pKW(x = c(1, 2, 3, 4, 5), g = c(1, 1, 2, 2, 2))
```

We try the second approach with our data - it is our usual format with one variable being the response and the second being the group variable. Normally, that is a factor variable. Here it seems to work the same whether we feed in the categorical variable or the numerically coded version.

```
set.seed(225)
with(airquality, pKW(x = Ozone, g = factor(Month)))
```

```
## Ties are present, so p-values are based on conditional null distribution.
## Group sizes: 26 9 26 26 29
## Kruskal-Wallis H Statistic: 29.2516
## Monte Carlo (Using 10000 Iterations) upper-tail probability: 0
```

```
set.seed(225)
# equivalent command without using *with* syntax
pKW(x = airquality$Ozone, g = factor(airquality$Month))
```

```
## Ties are present, so p-values are based on conditional null distribution.
## Group sizes: 26 9 26 26 29
## Kruskal-Wallis H Statistic: 29.2516
## Monte Carlo (Using 10000 Iterations) upper-tail probability: 0
```

What conclusions would you make here? Are there lingering concerns about any assumptions?

An alternative and simpler way to run the test is to use the `kruskal.test` command.

```
kruskal.test(Ozone ~ Monthf, data = airquality)
```

```
##
## Kruskal-Wallis rank sum test
##
## data: Ozone by Monthf
## Kruskal-Wallis chi-squared = 29.267, df = 4, p-value = 6.901e-06
```

This appears to jump to the large-sample approximation pretty quickly (not sure it does exact computations at all, actually), as it provides a chi-squared statistic, but this is very close to the reported H from `pKW`.

Do the two different implementations of the test agree in terms of their conclusion?

Nonparametric Multiple Comparisons

Assuming we are okay with the conditions for inference, we've detected a difference in medians. Where? We use nonparametric multiple comparisons to figure that out. We'll look at the book approach, along with other ways that this is coded in R.

Hollander, Wolfe, and Chicken Approach

We want to do pairwise Wilcoxon Rank Sum tests (Mann-Whitney U) and use a critical value based on the `cSDCFlig` function. This does Monte Carlo simulations - so you want to set a seed.

```
set.seed(225)
cSDCFlig(0.05, c(26, 9, 26, 26, 29))
```

```
##
## Monte Carlo Approximation (with 10000 Iterations) used:
##
## Group sizes: 26 9 26 26 29
## For the given experimentwise alpha=0.05, the upper cutoff value is Dwass, Steel, Critchlow-Fligner W
```

It does a lot of computation, so this command can take a while. Here, we find $\alpha = 0.0495$ and the cutoff value is $W_{\text{star}} = 3.80462$.

Now, we need to GET the W's for all pairwise combinations of groups, so that we can standardize them to W_{star} 's. Remember that we really get U? Well, we have to convert to W again, and then to W_{star} . We'll show this for a few pairs, before showing you other ways to do this in R.

```
with(airquality, wilcox.test(Ozone[Monthf == 5], Ozone[Monthf == 6]))

## Warning in wilcox.test.default(Ozone[Monthf == 5], Ozone[Monthf == 6]): cannot
## compute exact p-value with ties
##
## Wilcoxon rank sum test with continuity correction
##
## data: Ozone[Monthf == 5] and Ozone[Monthf == 6]
## W = 82, p-value = 0.1925
## alternative hypothesis: true location shift is not equal to 0
m <- 26 # size of first group
n <- 9 # size of second group
Uprime <- 82
U <- m*n - Uprime
W <- U + n*(n+1)/2; W # this is the W

## [1] 197

Wstar <- (W - 0.5*(n*(m+n+1)))/sqrt(m*n*(m+n+1)/24); Wstar

## [1] 1.868162
```

W_{star} here is 1.86816, which is less than the cutoff, so we do not have evidence of a significant difference in Ozone level medians between Months 5 and 6.

Here is another comparison:

```
with(airquality, wilcox.test(Ozone[Monthf == 5], Ozone[Monthf == 7]))

## Warning in wilcox.test.default(Ozone[Monthf == 5], Ozone[Monthf == 7]): cannot
## compute exact p-value with ties
##
## Wilcoxon rank sum test with continuity correction
##
## data: Ozone[Monthf == 5] and Ozone[Monthf == 7]
## W = 109.5, p-value = 2.996e-05
## alternative hypothesis: true location shift is not equal to 0
m <- 26 # size of first group
n <- 26 # size of second group
Uprime <- 109.5
U <- m*n - Uprime
W <- U + n*(n+1)/2; W # this is the W

## [1] 917.5

Wstar <- (W - 0.5*(n*(m+n+1)))/sqrt(m*n*(m+n+1)/24); Wstar

## [1] 5.913989
```

W_{star} here is 5.91399, which is above the cutoff of 3.7787. Based on our evidence, there is a significant difference in medians between Months 5 and 7 in terms of Ozone levels.

For five groups, there are 10 possible pairwise comparisons (calculation shown). Can we do this faster?

```
choose(5, 2)
```

```
## [1] 10
```

Hollander, Wolfe, and Chicken Approach - Automated

Luckily, we have the approach automated for us with `pSDCFlig`. It gives the p-values for the related comparisons, after adjusting. The function has the same structure as `pKW`. It does some simulating, so it can take a while to run. If it's taking a LONG time to run (say > 15 minutes), you can tweak the `n.mc` option which governs the number of Monte Carlo simulations. The default is 10000, which is why it can take a while.

```
set.seed(225)
# pSDCFlig(x = airquality$Ozone, g = airquality$Monthf) # should be equivalent
with(airquality, pSDCFlig(Ozone, Monthf))
```

```
## Ties are present, so p-values are based on conditional null distribution.
## Group sizes: 26 9 26 26 29
## Using the Monte Carlo (with 10000 Iterations) method:
##
## For treatments 5 - 6, the Dwass, Steel, Critchlow-Fligner W Statistic is 1.8697.
## The smallest experimentwise error rate leading to rejection is 0.6855 .
##
## For treatments 5 - 7, the Dwass, Steel, Critchlow-Fligner W Statistic is 5.9155.
## The smallest experimentwise error rate leading to rejection is 0 .
##
## For treatments 5 - 8, the Dwass, Steel, Critchlow-Fligner W Statistic is 5.4499.
## The smallest experimentwise error rate leading to rejection is 3e-04 .
##
## For treatments 5 - 9, the Dwass, Steel, Critchlow-Fligner W Statistic is 2.2191.
## The smallest experimentwise error rate leading to rejection is 0.5136 .
##
## For treatments 6 - 7, the Dwass, Steel, Critchlow-Fligner W Statistic is 3.4969.
## The smallest experimentwise error rate leading to rejection is 0.0849 .
##
## For treatments 6 - 8, the Dwass, Steel, Critchlow-Fligner W Statistic is 3.177.
## The smallest experimentwise error rate leading to rejection is 0.1518 .
##
## For treatments 6 - 9, the Dwass, Steel, Critchlow-Fligner W Statistic is -0.0972.
## The smallest experimentwise error rate leading to rejection is 1 .
##
## For treatments 7 - 8, the Dwass, Steel, Critchlow-Fligner W Statistic is -0.2589.
## The smallest experimentwise error rate leading to rejection is 0.9999 .
##
## For treatments 7 - 9, the Dwass, Steel, Critchlow-Fligner W Statistic is -4.7817.
## The smallest experimentwise error rate leading to rejection is 0.0041 .
##
## For treatments 8 - 9, the Dwass, Steel, Critchlow-Fligner W Statistic is -4.1743.
## The smallest experimentwise error rate leading to rejection is 0.0191 .
##
```

This confirms the results we had for the 2 examples above AND gives p-values (those are the error rates - the smallest error rate at which you could reject is the p-value). So, just set an overall level of significance and compare the p-values to that, or compare the critical value from `cSDCFlig` to the statistics here.

Remember, for both `SDCFlig` functions, it is useful to set a seed for reproducibility, and be aware that you

can ‘cache’ results as an option in the `{r}` chunk options, if things are taking a while to run/compile.

Other Automated Approaches

There are many other nonparametric methods out there. Here are just a few.

First, we could just run all of the Wilcoxon Rank Sum tests pairwise and report their p-values. This can be done with p-value adjustments for multiple testing or not. Here, we show it with no adjustment first. You’ll notice the 5-6 and 5-7 comparison p-values match those from above. This is currently un-adjusted, but the whole point of using `Wstars` was to adjust for multiple testing.

```
with(airquality, pairwise.wilcox.test(Ozone, Monthf, p.adjust.method = "none"))

##
## Pairwise comparisons using Wilcoxon rank sum test with continuity correction
##
## data: Ozone and Monthf
##
##      5      6      7      8
## 6 0.19250 -      -      -
## 7 3e-05  0.01414 -      -
## 8 0.00012 0.02591 0.86195 -
## 9 0.11859 0.95887 0.00074 0.00325
##
## P value adjustment method: none
```

So, let’s adjust. There are many methods you could call here, but Holm’s method is a reasonable one - it’s better than Bonferroni (and not quite as conservative).

```
with(airquality, pairwise.wilcox.test(Ozone, Monthf, p.adjust.method = "holm"))

##
## Pairwise comparisons using Wilcoxon rank sum test with continuity correction
##
## data: Ozone and Monthf
##
##      5      6      7      8
## 6 0.5775 -      -      -
## 7 0.0003 0.0848 -      -
## 8 0.0011 0.1295 1.0000 -
## 9 0.4744 1.0000 0.0060 0.0227
##
## P value adjustment method: holm
```

You can look up options for `p.adjust` to see other adjustments to try.

Another method is Dunn’s test. The function to run it is in another package - `dunn.test`. This appears to run the Kruskal-Wallis procedure AND do the comparisons. This test is for what is called “stochastic dominance” - you can read about it in the help file for the function. Basically, what you are comparing here are the CDFs of the two groups. Suppose the first group has CDF F_1 and second group has CDF F_2 . The procedure is testing whether $F_1(y)$ is less than $F_2(y)$ (or vice versa) for all y . This isn’t to say that the two distributions can’t overlap. It does say that CDFs can’t cross.

Here is the test with no adjustment.

```
# requires dunn.test package
# note: not rendering output due to LaTeX error with font
with(airquality, dunn.test(x = Ozone, g = Monthf))
```

Here is an adjustment to the p-values. Again, there are several options available.

```
with(airquality, dunn.test(x = Ozone, g = Monthf, method = "holm"))
```

When running this code yourself, take a look at the difference here between unadjusted and adjusted p-values. This is why we need to make the adjustments!

One final method is Nemenyi's test. This method may not work as well if the groups have unequal sample sizes. It's in the *DescTools* library, which we loaded above for parametric multiple comparisons.

```
# other dist option is chisq
with(airquality, NemenyiTest(x = Ozone, g = Monthf, dist = "tukey"))

##
## Nemenyi's test of multiple comparisons for independent samples (tukey)
##
##      mean.rank.diff      pval
## 6-5      12.02991453 0.88737
## 7-5      41.21153846 9.7e-05 ***
## 8-5      38.53846154 0.00035 ***
## 9-5      11.99734748 0.67819
## 7-6      29.18162393 0.16373
## 8-6      26.50854701 0.24773
## 9-6      -0.03256705 1.00000
## 8-7      -2.67307692 0.99853
## 9-7     -29.21419098 0.01136 *
## 9-8     -26.54111406 0.02867 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

These are ALL nonparametric methods based on rank sums. But they work slightly differently, and you can see different p-values as a result. The idea is that we hope there is overall consensus about which medians are significantly different.

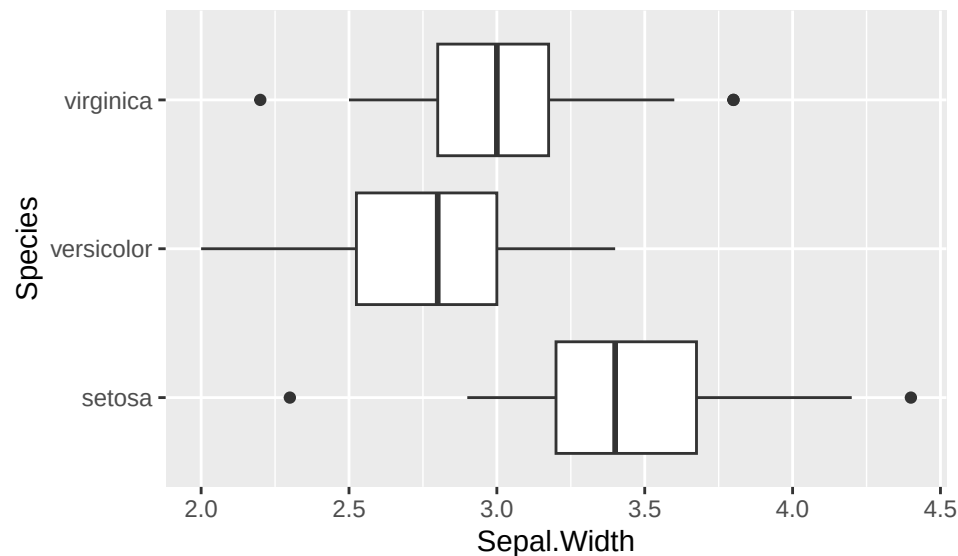
Another Example

The *airquality* data was an example with several issues. It was an unbalanced design, with missing data, and it was likely that the condition for equal spreads was not met. There may also have been issues with independence.

So, let's look at a more “well-behaved” example.

```
data(iris)
names(iris)

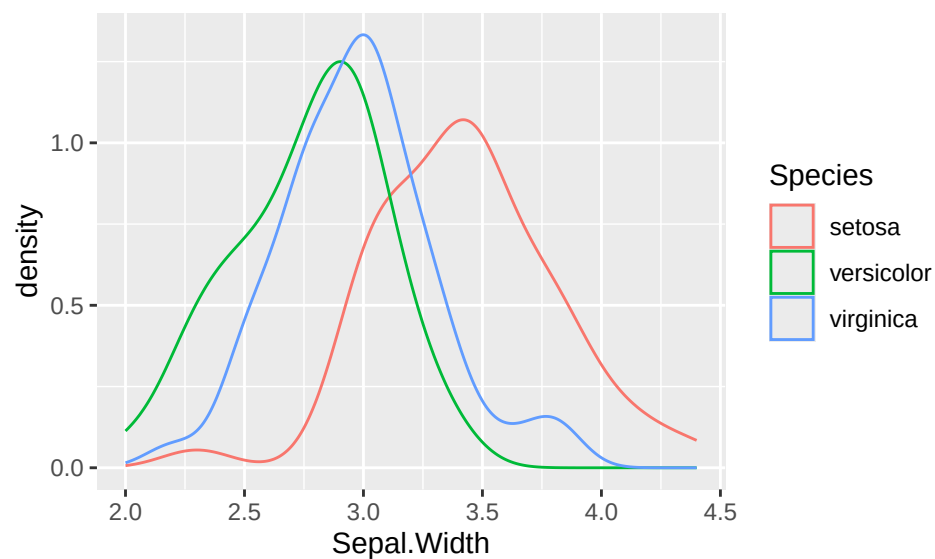
## [1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"
ggplot(aes(x = Sepal.Width, y = Species), data = iris) + geom_boxplot()
```



```
favstats(Sepal.Width ~ Species, data = iris)
```

```
##      Species min    Q1 median    Q3 max  mean      sd  n missing
## 1   setosa  2.3 3.200    3.4 3.675 4.4 3.428 0.3790644 50      0
## 2 versicolor 2.0 2.525    2.8 3.000 3.4 2.770 0.3137983 50      0
## 3  virginica 2.2 2.800    3.0 3.175 3.8 2.974 0.3224966 50      0
```

```
ggplot(aes(x = Sepal.Width, color = Species), data = iris) + geom_density()
```



How can we tell this design is balanced?

Do the conditions for ANOVA or Kruskal-Wallis appear to be satisfied based on these plots?

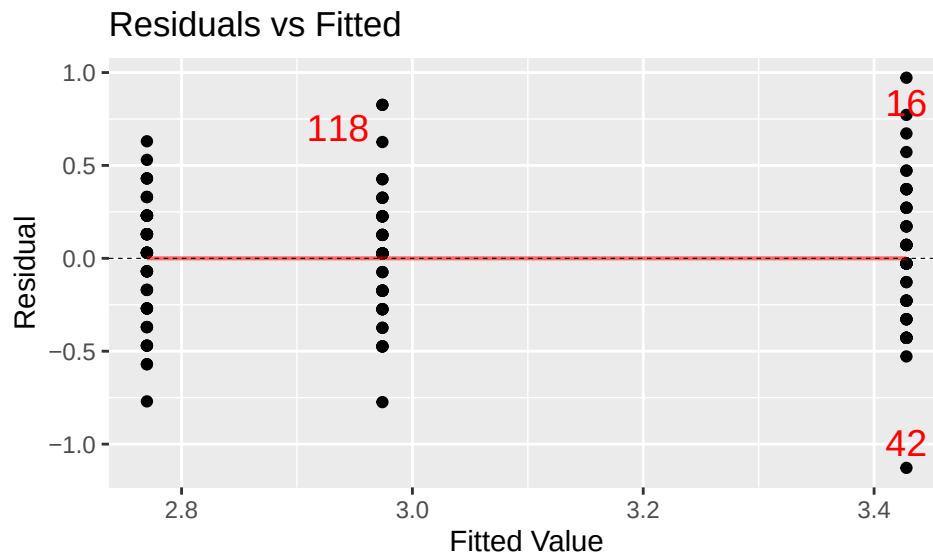
```
mod2 <- lm(Sepal.Width ~ Species, data = iris)
anova(mod2)
```

```
## Analysis of Variance Table
##
## Response: Sepal.Width
```

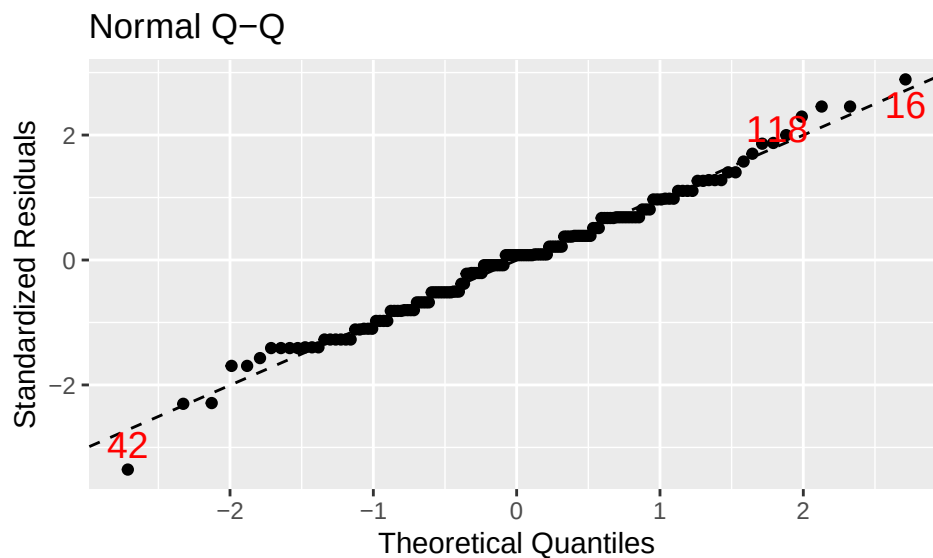
```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## Species      2 11.345   5.6725   49.16 < 2.2e-16 ***
## Residuals 147  16.962   0.1154
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
mplot(mod2, which = 1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
mplot(mod2, which = 2)
```



Based on the output, what conclusion would you reach for the ANOVA?

Based on the output, would it be necessary to swap to Kruskal-Wallis here or are the conditions for ANOVA satisfied?

```
set.seed(225)
with(iris, pKW(Sepal.Width, Species))
```

```
## Ties are present, so p-values are based on conditional null distribution.
## Group sizes: 50 50 50
## Kruskal-Wallis H Statistic: 63.0312
## Monte Carlo (Using 10000 Iterations) upper-tail probability: 0
```

```
# pKW(iris$Sepal.Width,iris$Species) # equivalent command
kruskal.test(Sepal.Width ~ Species, data = iris)
```

```
##
## Kruskal-Wallis rank sum test
##
## data: Sepal.Width by Species
## Kruskal-Wallis chi-squared = 63.571, df = 2, p-value = 1.569e-14
```

How similar are the pKW H and Kruskal.test chi-squared statistics here?

What conclusion do you reach about whether or not the medians are different between the iris Species for Sepal.Width?

Nonparametric Multiple Comparisons

We want to do pairwise Wilcoxon Rank Sum tests and use a critical value based on the `cSDCFlig` function.

```
set.seed(225)
cSDCFlig(0.05, c(50, 50, 50))
```

```
##
## Monte Carlo Approximation (with 10000 Iterations) used:
##
## Group sizes: 50 50 50
## For the given experimentwise alpha=0.05, the upper cutoff value is Dwass, Steel, Critchlow-Fligner W
Here, we find alpha = 0.05 and the cutoff value is Wstar = 3.30502.
```

One example by hand again - 3 groups is only 3 comparisons, but the automated approach is next.

```
with(iris, wilcox.test(Sepal.Width[Species == "versicolor"],
                      Sepal.Width[Species == "virginica"]))
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: Sepal.Width[Species == "versicolor"] and Sepal.Width[Species == "virginica"]
## W = 841, p-value = 0.004572
## alternative hypothesis: true location shift is not equal to 0
```

```
m <- 50 # size of first group
n <- 50 # size of second group
Uprime <- 841
U <- m*n-Uprime
W <- U+n*(n+1)/2; W # this is the W
```

```
## [1] 2934
```

```
Wstar <- (W-0.5*(n*(m+n+1)))/sqrt(m*n*(m+n+1)/24); Wstar
```

```
## [1] 3.987477
```

Wstar here is 3.98748, which is above the cutoff, so we have evidence of a significant difference in *Sepal.Width* medians between *versicolor* and *virginica* irises.

The automated approach gives:

```
set.seed(225)
with(iris, pSDCFlig(Sepal.Width, Species))

## Ties are present, so p-values are based on conditional null distribution.
## Group sizes: 50 50 50
## Using the Monte Carlo (with 10000 Iterations) method:
##
## For treatments setosa - versicolor, the Dwass, Steel, Critchlow-Fligner W Statistic is -10.3846.
## The smallest experimentwise error rate leading to rejection is 0 .
##
## For treatments setosa - virginica, the Dwass, Steel, Critchlow-Fligner W Statistic is -8.1911.
## The smallest experimentwise error rate leading to rejection is 0 .
##
## For treatments versicolor - virginica, the Dwass, Steel, Critchlow-Fligner W Statistic is 4.0152.
## The smallest experimentwise error rate leading to rejection is 0.0135 .
##
```

Again, values may differ a little due to simulations done here with the presence of ties in order to get the conditional null distribution.

Simulating the Null distribution of H for Krusal-Wallis

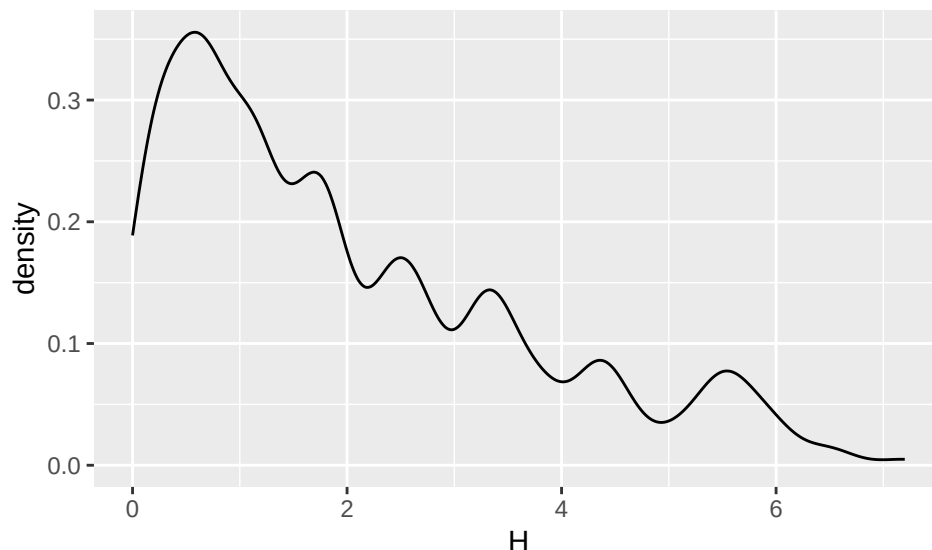
In the text, in the comments they go through the case of 3 groups with 2 observations in each group and develop the null distribution of H. There are 90 possible rank assignments in that case. So clearly, we don't want to try listing out values here.

Here is code for a short simulation to generate the null distribution of H for the setting with $k=3$, with three observations per group. Note that this will be an approximate distribution, because we are using repeated sampling to generate the distribution, rather than listing all possible rank assignments.

```
ranks <- c(1:9) # list the possible ranks
group <- c(rep(1:3, each = 3)) # set possible groups - 3 of each
N <- 9
runs <- 10000
H <- rep(0, runs)

set.seed(225)
for(i in 1:runs){
  grouprun <- sample(group, 9, replace = FALSE) # shuffles groups
  gmeans <- mean(ranks ~ grouprun) # gets mean ranks per group
  H[i] <- (12/(N*(N+1)))*3*sum((gmeans-(N+1)/2)^2) # computes H and stores
}
Hdata <- data.frame(H)

ggplot(data = Hdata, aes(x = H)) + geom_density()
```



```
favstats(~ H, data = Hdata)
```

```
##   min      Q1   median  Q3 max    mean      sd    n missing
##    0 0.622222 1.688889 3.2 7.2 2.009147 1.646306 10000      0
```

Describe the null distribution of H.

What would you need to change in the code if there were FOUR observations per group? (Hint: there are several places where values need adjusted).

What would you need to change in the code if there were ties in the ranks? (Hint: there is just one place you'd need to adjust values).

Data Sources

Data sets in these examples are included with the R software.

Airquality data. Available in R with `data(airquality)`. Help references Chambers, J. M., Cleveland, W. S., Kleiner, B. and Tukey, P. A. (1983) Graphical Methods for Data Analysis. Belmont, CA: Wadsworth.

Iris data. Available in R with `data(iris)`. Original sources referenced in R help:

Fisher, R. A. (1936) The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7, Part II, 179–188. doi:10.1111/j.1469-1809.1936.tb02137.x.

Anderson, Edgar (1935). The irises of the Gaspe Peninsula, *Bulletin of the American Iris Society*, 59, 2–5.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to *Nonparametric Statistical Methods* (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.